

原始文本语料
data/train.txt
(每行一段,空行跳过)

BPETokenizer.encode(行文本)
逐行编码 + 追加 <eos>(id=0)
→ List[int] token IDs

TextDataset.from_file()
拼接成单条长 token 流
np.uint32 零拷贝数组
长度 = 总 token 数

TextDataset.random_batch()
随机采样 batch_size 个起点
截取长度 seq_len+1 窗口
x = window[:-1], y = window[1:]
→ x[B,S], y[B,S] (右移一位)

关键设计:

- 语料拼接成单条流,随机截窗
- y = x 右移一位 = Next-Token 目标
- <eos> 分隔文档,模型学会重置
- 梯度累积等效大 batch,省显存
- Loss 尖峰检测跳过异常更新
- RNG 状态保存实现完全可复现续训

model.forward(x)
LLMModel 前向传播
Embedding → N×TransformerBlock
→ Norm → LM Head
→ logits [B, S, V]

F.cross_entropy(logits, y)
logits.view(-1,V), y.view(-1)
Next-Token Prediction Loss
ignore_index=-1 (预留padding)

loss.backward()
梯度累积 gradient_accumulation_steps
clip_grad_norm_(max_norm=1.0)
AdamW.step() + lr_schedule

save_checkpoint()
model.pt (权重) + meta.json (配置)
optim.pt (优化器状态 + RNG)
best/ 仅保存最优 val_loss 权重

下一 step